

Prime Tech Systems

Lógica de Desarrollo

Documentación técnica del proyecto

Jose Luis Perez
1º DAM — Módulo: Programación
Mayo 2026

Índice de contenidos

1. Arranque de la aplicación — PrimeTech.java	3
2. Pantalla de carga — PantallaCarga y HiloCarga	4
3. Gestión de conexiones — ConexionDB	5
4. Sistema de sesión — Sesion	6
5. Login — LoginFrame	7
6. Patrón DAO	8
7. Modelos de datos	10
8. Pantalla principal — Pantalla.java	11
9. Navegación entre paneles	12
10. Timer de inactividad	13
11. Estilo visual de las tablas	13
12. Búsqueda en tiempo real	15
13. Dashboard y estadísticas	15
14. Sistema de facturas y QR	16
15. Exportación a Excel	17
16. Seguridad	17
17. CI/CD con GitHub Actions	18

1. Arranque de la aplicación — PrimeTech.java

El punto de entrada de toda la aplicación es la clase **PrimeTech.java**. Antes de abrir ninguna ventana configuro FlatLaf y todos los estilos visuales globales mediante **UIManager.put()**. Es imprescindible hacerlo aquí, antes de instanciar cualquier componente Swing, porque FlatLaf aplica los estilos en el momento en que se crea cada componente. Si se configurara después ya estarían creados con los valores por defecto.

Los estilos que configuro globalmente son:

- Botones redondeados: `Button.arc = 999` (cuanto mayor el valor, más redondo)
- Botones en cian [0, 204, 255] con texto blanco y hover más oscuro [0, 170, 210]
- Fuente Roboto 13pt en filas de tabla y Roboto Bold 13pt en cabeceras
- Cabecera de tabla en azul pizarra [50, 90, 150] con texto blanco
- Selección de filas en cian [0, 204, 255] con texto blanco
- Scrollbar discreta en gris oscuro [80, 80, 100]
- Sin borde de foco al hacer clic (`Component.focusWidth = 0`)

Después lanzo la pantalla de carga dentro de **SwingUtilities.invokeLater()**, que garantiza que todo lo visual se ejecuta en el hilo de Swing (EDT — Event Dispatch Thread). Dentro de ese bloque creo la **PantallaCarga** y arranco el **HiloCarga**:

```
SwingUtilities.invokeLater(() -> { PantallaCarga splash = new PantallaCarga();
splash.setLocationRelativeTo(null); // centrar en pantalla splash.setVisible(true); //
forma corta: creamos el hilo y lo arrancamos en la misma línea // no guardamos la
referencia porque no necesitamos pausarlo ni cancelarlo new HiloCarga(splash).start();
});
```

He usado la forma corta **new HiloCarga(splash).start()** sin guardar el objeto en una variable porque en este caso no necesito controlarlo después de arrancarlo. Si necesitara pausarlo, cancelarlo o comprobar si sigue vivo, guardaría la referencia para poder llamar a `hilo.interrupt()` o `hilo.isAlive()`.

2. Pantalla de carga — PantallaCarga y HiloCarga

La pantalla de carga es un **JFrame** sin decoración (`undecorated = true`) con tres elementos: el logo de la tienda, una barra de progreso (`jProgressBar1`) y un **JLabel** (`lblEstado`) para el texto de estado.

2.1 Método `setProgreso()`

La clase **PantallaCarga** expone un método público **setProgreso(int valor, String mensaje)** que actualiza tanto la barra como el label. Este método puede ser llamado desde cualquier hilo porque internamente usa `SwingUtilities.invokeLater()` para devolver la actualización visual al EDT:

```
public void setProgreso(int valor, String mensaje) { SwingUtilities.invokeLater(() -> {
jProgressBar1.setValue(valor); lblEstado.setText(mensaje); }); }
```

2.2 HiloCarga — lógica del hilo

HiloCarga extiende **Thread** y recibe la **PantallaCarga** en su constructor. Al llamar a `start()`, Swing lanza el método `run()` en un hilo separado al EDT. Esto es fundamental: si hiciera la conexión a la base de datos

en el EDT, la pantalla de carga se congelaría y la barra no avanzaría.

```
public class HiloCarga extends Thread { private final PantallaCarga splash; public
HiloCarga(PantallaCarga splash) { this.splash = splash; } @Override public void run() {
try { splash.setProgresso(20, "Iniciando..."); Thread.sleep(400); splash.setProgresso(50,
"Conectando con la base de datos..."); ConexionDB.getConnectionApp().close(); // fuerza
init del pool HikariCP Thread.sleep(400); splash.setProgresso(80, "Cargando
interfaz..."); Thread.sleep(400); splash.setProgresso(100, "Listo"); Thread.sleep(300);
SwingUtilities.invokeLater(() -> { splash.dispose(); new LoginFrame().setVisible(true);
}); } catch (Exception e) { e.printStackTrace(); } } }
```

Uso **getConnectionApp()** y no **getConnection()** porque este último consulta la sesión para saber si el empleado es admin, pero en este momento todavía no hay nadie logueado. **getConnectionApp()** no depende de la sesión y siempre funciona. El **.close()** al final devuelve la conexión al pool inmediatamente — solo la pedimos para forzar que HikariCP inicialice el pool, no para ejecutar ninguna consulta.

3. Gestión de conexiones — ConexionDB

ConexionDB gestiona dos pools de conexiones HikariCP que se inicializan una única vez cuando la JVM carga la clase, gracias al bloque static {}. La configuración (URL, usuario, contraseña) se lee de **config.properties**, que está dentro del JAR y nunca se sube al repositorio.

3.1 Dos pools con permisos distintos

- **poolApp** — usuario **tienda_app**: solo SELECT, INSERT, UPDATE y DELETE. Para empleados normales.
- **poolAdmin** — usuario **tienda_admin**: todos los privilegios. Para administradores.
- **minimumIdle** = 0 en ambos pools: no mantiene conexiones abiertas cuando no hay actividad.
- **maxPoolSize** = 5 para app y 3 para admin: limita el número máximo de conexiones simultáneas.

```
static { // se ejecuta una sola vez al cargar la clase Properties props =
cargarConfig(); HikariConfig cfgApp = new HikariConfig();
cfgApp.setUsername(props.getProperty("db.usuario.app")); cfgApp.setMaximumPoolSize(5);
cfgApp.setMinimumIdle(0); // sin conexiones abiertas en reposo poolApp = new
HikariDataSource(cfgApp); HikariConfig cfgAdmin = new HikariConfig();
cfgAdmin.setUsername(props.getProperty("db.usuario.admin"));
cfgAdmin.setMaximumPoolSize(3); cfgAdmin.setMinimumIdle(0); poolAdmin = new
HikariDataSource(cfgAdmin); }
```

3.2 Selección de pool según sesión

El método **getConnection()** decide qué pool usar en función del empleado logueado. Si no hay sesión activa o el cargo no es admin, devuelve una conexión del **poolApp**:

```
public static Connection getConnection() throws SQLException { if (Sesion.haySession() &&
"admin".equals(Sesion.getEmpleado().getCargo())) { return getConnectionAdmin(); } return
getConnectionApp(); }
```

4. Sistema de sesión — Sesion

Sesion es una clase utilitaria con un único campo estático empleadoActual. Funciona como registro global del empleado que está usando la aplicación en ese momento. Al ser estático, cualquier clase puede consultarlo sin necesidad de pasar el empleado como parámetro.

- iniciar(Empleado emp) — guarda el empleado al hacer login correctamente
- cerrar() — pone empleadoActual a null al cerrar sesión o por inactividad
- getEmpleado() — devuelve el empleado actual
- haySession() — devuelve empleadoActual != null
- esAdmin() — comprueba que hay sesión y que el cargo es 'admin'

Gracias a esta clase, ConexionDB puede decidir qué pool usar en tiempo real sin necesidad de que nadie le pase el empleado como parámetro en cada llamada.

5. Login — LoginFrame

LoginFrame es un JFrame sin decoración del sistema operativo (undecorated = true) con una cabecera personalizada que incluye botones de cerrar y minimizar. Para que la ventana sea arrastrable, añadido en el constructor listeners de ratón sobre el panel header que calculan el desplazamiento y mueven la ventana:

```
final int[] posInicial = new int[2]; header.addMouseListener(new MouseAdapter() { public  
void mousePressed(MouseEvent e) { posInicial[0] = e.getXOnScreen(); posInicial[1] =  
e.getYOnScreen(); } }); header.addMouseMotionListener(new MouseMotionAdapter() { public  
void mouseDragged(MouseEvent e) { int dx = e.getXOnScreen() - posInicial[0]; int dy =  
e.getYOnScreen() - posInicial[1]; setLocation(getLocation().x + dx, getLocation().y +  
dy); posInicial[0] = e.getXOnScreen(); posInicial[1] = e.getYOnScreen(); } });
```

5.1 Proceso de login

- El usuario introduce email y contraseña
- EmpleadoDAO verifica con BCrypt.checkpw() que la contraseña coincide con el hash en BD
- Si falla: incrementa intentosFallidos en BD. Al llegar a 5, bloquea la cuenta
- Si la cuenta está bloqueada: muestra mensaje específico sin incrementar el contador
- Si tiene éxito: reinicia intentosFallidos, llama a Sesion.iniciar(empleado), abre Pantalla
- El lblError muestra el mensaje de error y desaparece automáticamente tras 4 segundos

Los botones btnCerrarLogin y btnMinimizarLogin tienen el fondo puesto a blanco directamente en el constructor (no en initComponents) para sobrescribir el cian que FlatLaf aplica globalmente a todos los botones.

6. Patrón DAO

Cada entidad del sistema tiene su propia clase DAO que concentra todas las consultas SQL. Los DAOs obtienen una conexión del pool de HikariCP, ejecutan la consulta y cierran la conexión automáticamente usando try-with-resources. Todos usan Log4j2 para registrar las operaciones en el archivo de log.

En Pantalla.java, los DAOs se instancian una sola vez como variables de clase para no crear un objeto nuevo en cada operación:

```
private ProductoDAO productoDAO = new ProductoDAO(); private VentaDAO ventaDAO = new
VentaDAO(); private ClienteDAO clienteDAO = new ClienteDAO();
```

6.1 EmpleadoDAO

- login(): busca el empleado por email, verifica contraseña con BCrypt.checkpw()
- incrementarIntentos(): suma 1 a intentosFallidos en la BD
- bloquearCuenta(): pone bloqueado = true cuando intentosFallidos llega a 5
- resetearIntentos(): pone intentosFallidos = 0 tras login exitoso

6.2 ProductoDAO

- listarTodos(): devuelve todos los productos ordenados por nombre
- insertar(), actualizar(), eliminar(): operaciones CRUD básicas
- buscarPorNombre(String texto): búsqueda con LIKE '%texto%' en nombre y descripción

6.3 ClienteDAO

- Soporta dos tipos de cliente: 'particular' y 'empresa'
- getNombreVisible() en el modelo devuelve nombre+apellidos para particular, razonSocial para empresa
- buscarPorNombre(): busca en nombre, apellidos y razón social simultáneamente
- CRUD completo con validación del tipo antes de insertar

6.4 VentaDAO

- registrarVenta(): inserta la venta y sus líneas de detalle en una transacción
- buscarPorCliente(): filtra ventas por nombre de cliente con LIKE
- ventasHoy(): suma el total de ventas del día actual
- ventasEsteMes(): cuenta el número de ventas del mes actual
- productoMasVendido(): consulta con GROUP BY y ORDER BY sobre líneas de detalle
- datosGraficaBarras(): devuelve ventas de los últimos 7 días agrupadas por fecha
- datosGraficaTarta(): devuelve los 5 productos con más unidades vendidas

7. Modelos de datos

Las clases del modelo representan las entidades del negocio y se usan para transportar datos entre los DAOs y la interfaz.

Clase	Campos principales
Empleado	id, nombre, apellidos, email, passwordHash, cargo, intentosFallidos, bloqueado
Cliente	id, tipo, nombre, apellidos, razonSocial, telefono, email — getNombreVisible() adapta según tipo
Producto	id, nombre, descripcion, precioCompra, precioVenta, stock

Venta	id, idCliente, nombreCliente, idEmpleado, fechaVenta (LocalDateTime), total, metodoPago, List<DetalleVenta>
DetalleVenta	idProducto, nombreProducto, cantidad, precioUnitario — getSubtotal() = cantidad * precioUnitario

8. Pantalla principal — Pantalla.java

Pantalla.java es el formulario principal de la aplicación. Contiene la lógica de gestión de productos, ventas, clientes y el dashboard. Para no crear objetos innecesarios, los DAOs se instancian como variables de clase.

8.1 Variables de control crear/editar

Uso el mismo diálogo tanto para crear como para editar registros. La variable de control actúa como indicador del modo actual:

```
private Producto productoActual = null; // null = crear, con valor = editar
private Cliente clienteActual = null;
private Venta ventaActual = null;
```

Al pulsar guardar, el código comprueba esta variable para llamar a insertar() o actualizar() del DAO correspondiente. Al abrir el diálogo para crear, se pone a null; al editar, se asigna el objeto seleccionado en la tabla.

8.2 Listas paralelas al combobox

En el diálogo de nueva venta hay comboboxes de clientes y productos. Mantengo listas paralelas **clientesCombo** y **productosCombo** sincronizadas con los comboboxes para poder recuperar el objeto completo (con su ID) cuando el usuario selecciona un ítem, ya que el combobox solo muestra el nombre.

9. Navegación entre paneles

En lugar de usar un JTabbedPane, la pantalla principal tiene cuatro paneles que ocupan el mismo espacio en la interfaz. Al iniciar la aplicación solo se muestra pnlProductos; los demás están ocultos con setVisible(false).

El método mostrarPanel() centraliza la lógica de navegación:

```
private void mostrarPanel(JPanel panel) {
    pnlProductos.setVisible(false);
    pnlVentas.setVisible(false);
    pnlClientes.setVisible(false);
    pnlDashboard.setVisible(false);
    panel.setVisible(true);
}
```

Los labels de navegación del sidebar llaman a este método en su evento mouseClicked. Esta solución me da control total sobre el diseño: puedo personalizar completamente el aspecto del sidebar sin las limitaciones visuales del JTabbedPane.

10. Timer de inactividad

Al abrir Pantalla se arranca un `javax.swing.Timer` configurado a 3 minutos (constante `MINUTOS_INACTIVIDAD`). El timer llama a `cerrarSesionPorInactividad()` si transcurren 3 minutos sin actividad.

- Cualquier movimiento de ratón o pulsación de teclado llama a `resetActivity()`
- `resetActivity()` reinicia el timer con `restart()`
- Al dispararse: `Sesion.cerrar()`, cierra Pantalla, abre un nuevo `LoginFrame`
- El timer usa `setRepeats(false)` para que solo se dispare una vez por ciclo de inactividad

11. Estilo visual de las tablas

El estilo de las tablas se construye en tres capas que trabajan juntas.

11.1 Capa 1 — UIManager global

- Fuente Roboto 13pt para filas, Roboto Bold 13pt para cabeceras
- Cabecera azul pizarra [50, 90, 150] con texto blanco
- Selección de filas en cian [0, 204, 255] con texto blanco
- Color de rejilla [210, 210, 220] entre filas con datos

11.2 Capa 2 — Viewport del JScrollPane

El área vacía debajo de los datos sale oscura [30, 30, 40] al poner ese color en el viewport del `JScrollPane`. Las celdas con datos no se ven afectadas porque el `renderer` tiene prioridad sobre el fondo del viewport.

11.3 Capa 3 — Renderers personalizados

Para ventas y clientes creo un `DefaultTableCellRenderer` que fuerza blanco en filas no seleccionadas:

```
new DefaultTableCellRenderer() { public Component getTableCellRendererComponent( JTable
table, Object value, boolean isSelected, boolean hasFocus, int row, int column) {
super.getTableCellRendererComponent(table, value, isSelected, hasFocus, row, column); if
(!isSelected) { setBackground(Color.WHITE); setForeground(new Color(30, 30, 40)); } //
si esta seleccionada, super() ya aplica el cian del UIManager return this; } }
```

Para productos el `renderer` aplica colores según el nivel de stock:

Stock	Color	Significado
< 5	Naranja [255, 180, 80]	Crítico — reponer urgente
5 – 9	Amarillo [255, 245, 130]	Bajo — avisar pronto
10 – 25	Blanco	Normal
> 25	Verde [180, 230, 180]	Bien abastecido

12. Búsqueda en tiempo real

Cada campo de búsqueda tiene un evento `keyReleased` que llama al método `buscar` del DAO en cada tecla que suelta el usuario. La tabla se recarga inmediatamente sin necesidad de pulsar `Enter` ni ningún botón.

Los botones de búsqueda originales se reconvirtieron en botones de limpiar: al pulsarlos, borran el texto del buscador y recargan la tabla completa. Así el flujo es más natural: escribes y filtra solo; pulsas el botón y vuelves a ver todos los registros.

```
// evento keyReleased de txtBuscarProducto try { DefaultTableModel m =
(DefaultTableModel) tblProductos.getModel(); m.setRowCount(0); for (Producto p :
productoDAO.buscarPorNombre(txtBuscarProducto.getText())) { m.addRow(new Object[]{
p.getIdProducto(), p.getNombre(), p.getDescripcion(), p.getPrecioCompra(),
p.getPrecioVenta(), p.getStock() }); } } catch (SQLException ex) {
JOptionPane.showMessageDialog(this, "Error: " + ex.getMessage()); }
```

13. Dashboard y estadísticas

El dashboard muestra estadísticas en tiempo real y dos gráficas generadas con `JFreeChart`:

- Estadísticas de texto: ventas de hoy en €, número de ventas del mes, producto más vendido
- Gráfica de barras: importe de ventas de los últimos 7 días, una barra por día
- Gráfica de tarta: los 5 productos más vendidos por unidades
- El dashboard se actualiza automáticamente al registrar o eliminar una venta

Los datos para las gráficas los proporciona `VentaDAO` mediante consultas SQL con `GROUP BY` y `ORDER BY`. `JFreeChart` recibe los datos y genera el panel gráfico que se añade directamente al panel del dashboard.

14. Sistema de facturas y QR

14.1 Generación de la factura HTML

`FacturaHTML` genera un archivo `.html` con diseño profesional que incluye: datos del cliente, datos del empleado que realizó la venta, fecha y hora, tabla de productos comprados con cantidades y precios, subtotales y total final.

14.2 Código QR

`GenerarQR` usa la librería `ZXing` para crear un código QR con el resumen de la compra. El QR se genera como imagen, se convierte a Base64 y se incrusta directamente en el HTML de la factura como una imagen inline. Así la factura es un único archivo HTML autocontenido que no necesita archivos externos.

14.3 Envío por email

`EnviarEmail` usa `Jakarta Mail` para enviar la factura HTML al cliente vía Gmail SMTP con TLS en el puerto 587. Las credenciales (correo y contraseña de aplicación) se leen de `email.properties`, que está en `.gitignore` y nunca se sube al repositorio.

15. Exportación a Excel

ExportarExcel usa Apache POI para generar un archivo .xlsx con tres hojas: Productos, Ventas y Clientes. Cada hoja tiene una fila de cabecera con estilo (fondo de color, texto en negrita) y todas las filas de datos actuales. El archivo se guarda en la ubicación que el usuario elija mediante un JFileChooser.

16. Seguridad

He aplicado varias capas de seguridad a lo largo de toda la aplicación:

Medida	Implementación
Contraseñas hasheadas	BCrypt con salt aleatorio. Nunca se almacena la contraseña en texto plano.
Dos usuarios de BD	tienda_app solo tiene DML (sin DDL). tienda_admin tiene todos los privilegios.
Bloqueo de cuenta	Después de 5 intentos fallidos la cuenta queda bloqueada en la BD.
Timeout de inactividad	La sesión se cierra automáticamente tras 3 minutos sin actividad.
Credenciales gitignoreadas	config.properties y email.properties están en .gitignore. Nunca se suben.
Pool de conexiones	HikariCP cierra conexiones inactivas y limita el número máximo simultáneo.

17. CI/CD con GitHub Actions

El repositorio tiene dos workflows configurados en .github/workflows/ que automatizan la compilación, los tests y las releases.

17.1 ci.yml — Tests en cada push

- Se dispara en cada push o Pull Request a la rama master
- Levanta un contenedor MySQL 8.0 con la base de datos y los usuarios necesarios
- Compila el proyecto con Ant y ejecuta los tests de integración
- Si algún test falla, el push queda marcado como fallido en GitHub

17.2 release.yml — Publicación automática

- Se dispara al crear un tag con formato vX.Y (por ejemplo: git tag v2.3 && git push origin v2.3)
- Compila con: ant jar -Djar.archive.disabled=false (necesario porque project.properties lo deshabilita)
- Copia exactamente los 20 JARs necesarios a dist/lib/ (sin extras que causen conflictos)
- Genera PrimeTech.bat y PrimeTech.sh con el flag --enable-native-access=ALL-UNNAMED para FlatLaf
- Empaqueta todo en un ZIP y lo publica en GitHub Releases via softprops/action-gh-release

- Requiere permissions: contents: write para poder crear la release

Los scripts de arranque usan **java -cp "PrimeTechFinal.jar;lib/*"** en lugar de **java -jar** porque el archivo MANIFEST.MF tiene un límite de 72 caracteres por línea que impide incluir todos los JARs del classpath. Con -cp se evita ese límite completamente.

```
# crear y publicar una nueva release git tag v2.3 git push origin v2.3 # GitHub Actions  
compila y publica el ZIP automáticamente
```